

TASK 1: PASSWORD SECURITY ENHANCEMENT

ORIGINAL INSECURE CODE

```
USERS = {}
```

```
DEF REGISTER(USERNAME, PASSWORD):
```

```
    USERS[USERNAME] = PASSWORD
```

```
WITH OPEN('USERS.TXT', 'A') AS F:
```

```
F.WRITE(F"{USERNAME}:{PASSWORD}\n")
```

```
DEF LOGIN(USERNAME, PASSWORD):
```

```
WITH OPEN('USERS.TXT', 'R') AS F:
```

```
FOR LINE IN F:
```

```
    U, P = LINE.STRIP().SPLIT(':')
```

```
IF U == USERNAME AND P == PASSWORD:
```

```
    RETURN True    RETURN False
```

```
REGISTER("TESTUSER", "PASSWORD123")
```

```
PRINT(LOGIN("TESTUSER",
```

```
"PASSWORD123"))
```

RAN IT:

TRUE

CHECKED USERS.TXT:

```
TESTUSER:PASSWORD123
```

RISK: PLAIN TEXT STORAGE, ANYONE WITH FILE ACCESS SEES
PASSWORDS.

REFACTORED WITH HASHLIB:

```
IMPORT
```

```
HASHLIB
```

```
IMPORT OS
```

```
DEF HASH_PASSWORD(PASSWORD):
```

```
    RETURN HASHLIB.SHA256(PASSWORD.ENCODE()).HEXDIGEST()
```

```
USERS = {}
```

```
DEF REGISTER(USERNAME, PASSWORD):
```

```
    HASHED = HASH_PASSWORD(PASSWORD)
```

```
    USERS[USERNAME] = HASHED    WITH OPEN('USERS.TXT',  
'A') AS F:
```

```
        F.WRITE(F"{USERNAME}:{HASHED}\n")
```

```
DEF LOGIN(USERNAME, PASSWORD):
```

```
    HASHED_INPUT = HASH_PASSWORD(PASSWORD)    IF USERNAME IN  
    USERS AND USERS[USERNAME] == HASHED_INPUT:    RETURN TRUE  
    WITH OPEN('USERS.TXT', 'R') AS F:    FOR LINE IN F:
```

```
        U, H = LINE.STRIP().SPLIT(':')    IF U == USERNAME  
        AND H == HASHED_INPUT:
```

```
            USERS[USERNAME] = H # CACHE    RETURN  
TRUE    RETURN FALSE
```

```
# TEST REGISTER("TESTUSER", "PASSWORD123")
```

```
PRINT(LOGIN("TESTUSER", "PASSWORD123")) # TRUE
```

```
PRINT(LOGIN("TESTUSER", "WRONGPASS")) # FALSE
```

OUTPUT:

TRUE

FALSE

USERS.TXT NOW HAS HASH:

TESTUSER:EF92B778BAFE771E89245B89ECBC08A44A4E166C06659
911881F383D4473E94F

TASK 2: SECURE API KEY MANAGEMENT

ORIGINAL:

```
IMPORT REQUESTS
```

```
API_KEY = "ABC123FAKEKEY" CITY = "LONDON" URL =  
F"HTTP://API.OPENWEATHERMAP.ORG/DATA/2.5/WEATHER?Q={CITY}  
&APPID={API_KEY}" RESPONSE = REQUESTS.GET(URL)  
PRINT(RESPONSE.JSON())
```

RISK: HARDCODED KEY EXPOSED IN GIT/SOURCE.

REFACTORED:

```
IMPORT REQUESTS IMPORT OS
```

```
API_KEY = OS.GETENV('WEATHER_API_KEY') IF NOT API_KEY:
```

```
    PRINT("SET WEATHER_API_KEY ENV VAR")    EXIT()
```

```
CITY = "LONDON" URL =
```

```
F"HTTP://API.OPENWEATHERMAP.ORG/DATA/2.5/WEATHER?Q={CITY}&APPID={API
```

```
RESPONSE = REQUESTS.GET(URL) IF
```

```
RESPONSE.STATUS_CODE == 200:
```

```
    PRINT(RESPONSE.JSON()['WEATHER']['^1_0']['DESCRIPTION']) ELSE:
```

```
    PRINT("API ERROR")
```

SET ENV IN TERMINAL:

```
EXPORT WEATHER_API_KEY="ABC123FAKEKEY"
```

```
PYTHON SCRIPT.PY
```

OUTPUT:

```
CLEAR SKY
```

NO KEY IN CODE NOW.

TASK 3: WEAK ENCRYPTION DETECTION

ORIGINAL WEAK (BASE64):

```
IMPORT BASE64
```

```
DEF ENCRYPT(TEXT):
```

```
    RETURN  
    BASE64.B64ENCODE(TEXT.ENCODE()).DECODE()
```

```
DEF DECRYPT(ENC):
```

```
    RETURN  
    BASE64.B64DECODE(ENC.ENCODE()).DECODE()
```

```
MSG  =  "SECRET"  ENC  =  
ENCRYPT(MSG)  
PRINT(DECRYPT(ENC))
```

OUTPUT:

```
SECRET
```

WEAKNESS: BASE64 NOT ENCRYPTION, JUST ENCODING, PUBLIC
REVERSIBLE.

IMPROVED WITH CRYPTOGRAPHY (FERNET):

```
FROM CRYPTOGRAPHY.FERNET
IMPORT FERNET

KEY =
FERNET.GENERATE_KEY()
CIPHER = FERNET(KEY)

DEF ENCRYPT(TEXT):
    RETURN
CIPHER.ENCRYPT(TEXT.ENCODE())

DEF DECRYPT(ENC):
    RETURN
CIPHER.DECRYPT(ENC).DECODE()

MSG = "SECRET" ENC =
ENCRYPT(MSG)
PRINT(DECRYPT(ENC))
PRINT(KEY.DECODE()) # SHARE
SECURELY
```

OUTPUT:

```
SECRET
GAAAAABK...== (EXAMPLE KEY)
```

WITHOUT KEY, CAN'T DECRYPT EASILY.

TASK 4: REAL-TIME APPLICATION SECURITY REVIEW (REST API)

GENERATED FLASK LOGIN API:

```
FROM FLASK IMPORT FLASK,
REQUEST, jsonify

APP =
FLASK(__NAME__)

USERS = {"ADMIN":
"ADMINPASS"}

@app.route('/login',
methods=['POST']) def
login():
    data = request.json    if
data['user'] in users and
users[data['user']] ==
data['pass']:
    return
jsonify({"status":
"OK"})    return
jsonify({"status":
"fail"}), 401

if __name__ == '__main__':
    app.run()
```

VULNS:

1. NO INPUT VALIDATION, JSON TRUST.
2. PLAIN TEXT COMPARE.
3. NO RATE LIMIT.

IMPROVED:

```
FROM FLASK IMPORT FLASK, REQUEST,  
JSONIFY IMPORT HASHLIB IMPORT TIME FROM  
FUNCTOOLS IMPORT WRAPS
```

```
APP = FLASK(__NAME__)
```

```
USERS = {"ADMIN":
```

```
HASHLIB.SHA256("ADMINPASS".ENCODE()).HEXDIGEST()}
```

```
FAILED_LOGINS = {}
```

```
DEF RATE_LIMIT(IP):  NOW = TIME.TIME()  IF IP NOT IN
FAILED_LOGINS:      FAILED_LOGINS[IP] = []
FAILED_LOGINS[IP] = [T FOR T IN FAILED_LOGINS[IP] IF
NOW - T < 60]  IF LEN(FAILED_LOGINS[IP]) > 5:
    RETURN FALSE  RETURN TRUE
```

```

DEF LOGIN_LIMITED(F): @WRAPS(F)
DEF DECORATED(*ARGS, **KWARGS):
    IP = REQUEST.REMOTE_ADDR
IF NOT RATE_LIMIT(IP):
    RETURN jsonify({"error": "RATE
LIMITED"}), 429    RETURN F(*ARGS, **KWARGS)
RETURN DECORATED

@app.route('/login', methods=['POST'])
DEF LOGIN(): IF NOT REQUEST.IS_JSON:
    RETURN jsonify({"error": "JSON
REQUIRED"}), 400    DATA = REQUEST.GET_JSON()
IF 'USER' NOT IN DATA OR 'PASS' NOT IN DATA:
    RETURN jsonify({"error": "MISSING FIELDS"}), 400
HASHED =
HASHLIB.SHA256(DATA['PASS'].encode()).hexdigest()
IF DATA['USER'] IN USERS AND USERS[DATA['USER']] ==
HASHED:
    FAILED_LOGINS[REQUEST.REMOTE_ADDR] = []
RETURN jsonify({"status": "OK"})
FAILED_LOGINS.setdefault(REQUEST.REMOTE_ADDR,
[]).append(time.time())    RETURN jsonify({"status":
"FAIL"}), 401

```

```
IF __NAME__ == '__MAIN__':  
APP.RUN(DEBUG=False)
```

TESTED WITH CURL:

```
CURL -X POST -H "CONTENT-TYPE: APPLICATION/JSON" -D  
'{"USER":"ADMIN","PASS":"ADMINPASS"}' HTTP://127.0.0.1:5000/LOGIN
```

```
{"STATUS":"OK"}
```

BAD PASS 6 TIMES: RATE LIMITED.

TASK 5: EXCEPTION HANDLING IMPROVEMENT

ORIGINAL ARITHMETIC:

```
A = INPUT("NUM1: ") B = INPUT("NUM2: ")
PRINT(A / B)
```

TEST INVALID:

NUM1: 10

NUM2: ABC

TRACEBACK (MOST RECENT CALL LAST):

FILE "...", LINE 3, IN <MODULE>

TYPEERROR: UNSUPPORTED OPERAND TYPE(S) FOR /: 'STR' AND 'STR'

ZERO:

NUM1: 10

NUM2: 0

TRACEBACK ... ZERODIVISIONERROR

IMPROVED:

WHILE

TRUE:

TRY:

```
A = FLOAT(INPUT("ENTER
FIRST NUMBER: ")) B =
```

```
FLOAT(INPUT("ENTER SECOND
NUMBER: ")) IF B == 0:
```

```
    RAISE
```

```
VALUEERROR("DIVISION BY ZERO
```

```
NOT ALLOWED")
```

```
PRINT(F"RESULT: {A / B}")
```

```
BREAK    EXCEPT VALUEERROR AS
```

```
E:
```

```
    PRINT(F"ERROR: {E}. PLEASE
```

```
ENTER VALID NUMBERS.")    EXCEPT
```

```
KEYBOARDINTERRUPT:
```

```
PRINT("\NEXITING.")
```

```
BREAK
```

TESTS:

VALID:

```
ENTER FIRST NUMBER: 10
```

```
ENTER SECOND NUMBER: 2
```

```
RESULT: 5.0
```

INVALID:

```
ENTER FIRST NUMBER: ABC
```

```
ERROR: COULD NOT CONVERT STRING TO FLOAT: 'ABC'. PLEASE ENTER  
VALID NUMBERS.
```

ZERO:

```
ENTER FIRST NUMBER: 10
```

ENTER SECOND NUMBER: 0

ERROR: DIVISION BY ZERO NOT ALLOWED. PLEASE ENTER VALID NUMBERS.